

JAVA

Streams em Java

Pipeline, operações intermediárias e terminais, Optional e Collectors — do zero, com exemplos do catálogo de uma livraria.

Introdução • Pipeline • Fontes • Filtros • Transformações • Collectors • Optional

8 capítulos • Série JavaStudiesHub

CONTEÚDO DA APOSTILA

- 01** Introdução a Streams
- 02** Pipeline e Estrutura
- 03** Fontes de Streams
- 04** Operações Intermediárias — Filtros
- 05** Operações Intermediárias — Transformações
- 06** Operações Terminais — Estatísticas e Matching
- 07** Operações Terminais — Transformações
- 08** Optional

JavaStudiesHub

Sumário

- 01 Introdução a Streams**
O que são, diferença de Collection e avaliação preguiçosa

- 02 Pipeline e Estrutura**
Source, operações intermediárias, terminal e otimização interna

- 03 Fontes de Streams**
Collection, Arrays, Stream.of, iterate, generate, IntStream e Map

- 04 Operações Intermediárias — Filtros**
distinct, filter, limit, skip, takeWhile e dropWhile

- 05 Operações Intermediárias — Transformações**
map, peek, sorted e streams primitivos

- 06 Operações Terminais — Estatísticas e Matching**
count, sum, average, summaryStatistics e allMatch/anyMatch/noneMatch

- 07 Operações Terminais — Transformações**
toList, toArray, collect, reduce, flatMap e Collectors

- 08 Optional**
Container para valores que podem estar ausentes

01

Introdução a Streams

O que são, diferença de Collection e avaliação preguiçosa

A documentação oficial descreve um Stream como uma sequência de elementos capaz de suportar operações agregadas, sequenciais ou paralelas. Antes de qualquer coisa, vale desfazer uma confusão comum de nomenclatura: os I/O streams do Java (`BufferedInputStream`, `FileOutputStream` e companhia) lidam com entrada e saída de bytes e não têm nenhuma relação direta com o assunto desta apostila. O Stream de dados que vamos estudar é uma API completamente diferente, voltada a processamento.

Outra forma de enxergar um Stream é como uma sequência de passos computacionais encadeados sobre um conjunto de dados — a mesma ideia de encadeamento que já aparece em lambdas, method references e nas interfaces funcionais como `Predicate` e `Function`. Streams pegam essa ideia e a transformam em um mecanismo formal e reutilizável de processamento de dados.

Streams não são Collections

`Collection` e `Stream` nascem de propósitos diferentes. Uma `Collection` existe para guardar e gerenciar elementos em memória — por isso seu vocabulário é `add`, `get`, `remove`, `contains`. Um `Stream`, em contraste, existe para descrever o processamento desses elementos, e não armazena nada: ele computa itens sob demanda a partir de uma fonte, que pode ser uma `Collection`, um array, um arquivo ou até um resultado de banco de dados. Repare que a interface `Stream` nem sequer declara métodos como `add` ou `get` — eles simplesmente não fazem parte do vocabulário de um `Stream`.

Collection	Stream
Armazena elementos em memória	Não armazena — computa sob demanda
Acesso direto (<code>get</code> , <code>add</code> , <code>remove</code>)	Opera sobre o conjunto como unidade (<code>filter</code> , <code>map</code> , <code>reduce</code>)
Pode ser percorrida várias vezes	Uso único — após a operação terminal, o pipeline se fecha
Mutação é permitida e comum	Processamento declarativo; a fonte original não é alterada

Não existe nada que um `Stream` faça que uma `Collection` não pudesse fazer com um laço tradicional. A diferença está no estilo: Streams aproximam o código Java de uma linguagem de consulta declarativa, parecida com SQL — você descreve o resultado desejado, e não o algoritmo passo a passo para chegar lá.

A relação com lambdas

Streams e expressões lambda foram desenhados um para o outro. As operações de `Stream` recebem, quase sempre, uma lambda ou uma referência a método como argumento. As quatro interfaces funcionais mais comuns nesse universo aparecem na tabela a seguir.

Interface	Método	Onde aparece em Streams
Predicate	boolean test(T t)	filter, takeWhile, dropWhile, allMatch...
Function	R apply(T t)	map, flatMap, collect
Consumer	void accept(T t)	forEach, peek
Supplier	T get()	Stream.generate

Avaliação preguiçosa (lazy evaluation)

Um detalhe que costuma surpreender quem vem de outras linguagens: chamar uma operação intermediária num Stream não dispara execução nenhuma. O pipeline fica pendente, guardado como uma receita a ser seguida, até que uma operação terminal seja finalmente invocada — só aí o processamento inteiro acontece de uma vez. É o mesmo comportamento de uma lambda isolada: o corpo existe, mas só roda quando alguém efetivamente chama o método funcional.

♦ Por que isso importa

A avaliação lazy dá à implementação de Stream liberdade para analisar o pipeline inteiro antes de processar qualquer elemento, podendo reordenar, fundir ou até descartar operações. Por isso, efeitos colaterais dentro de lambdas de operações intermediárias — como incrementar um contador externo — podem simplesmente nunca acontecer.

Por que adotar Streams

Três motivos resumem bem a proposta de valor dos Streams:

- tornam o processamento de dados uniforme e conciso, no espírito declarativo do SQL — você diz o que precisa acontecer, não como;
- ao lidar com grandes volumes, um stream paralelo entrega ganho real de performance com pouquíssima mudança de código;
- a combinação de Collections, lambdas e Streams costuma produzir o mesmo resultado com bem menos código e mais legibilidade.

Nada disso aposenta o código tradicional de Collections — iteradores, sort, subList continuam válidos; Streams são mais uma ferramenta na caixa, não uma substituição total.

02

Pipeline e Estrutura

Source, operações intermediárias, terminal e otimização interna

Chamamos de pipeline de stream a cadeia completa de operações executadas sobre um Stream. Imagine o catálogo de uma livraria: queremos pegar os primeiros 20 livros cadastrados, manter apenas os que pertencem aos gêneros Ficção ou Fantasia, formatar o título com o gênero na frente, ordenar alfabeticamente e imprimir o resultado.

```
catalogo.stream() // SOURCE
.limit(20) // Intermediária
.filter(l -> l.genero().equals("Ficção")
|| l.genero().equals("Fantasia")) // Intermediária
.map(l -> l.genero() + " - " + l.titulo()) // Intermediária
.sorted() // Intermediária
.forEach(System.out::println); // TERMINAL
```

Cada linha iniciada por ponto representa uma operação do pipeline. Colocar cada chamada em sua própria linha não é frescura estética — melhora demais a legibilidade de pipelines longos, e vale adotar como hábito permanente.

A fonte (source)

A fonte é de onde os elementos vêm — no exemplo acima, a lista catalogo. Como List e Set implementam Collection, o método `stream()` está disponível em qualquer implementação dela: ArrayList, LinkedList, HashSet, TreeSet e assim por diante. Existem muitas outras formas de criar uma fonte, incluindo streams infinitos, que veremos no próximo capítulo.

Operações intermediárias

Tudo o que fica entre a fonte e a operação terminal é uma operação intermediária. Elas não são obrigatórias — um pipeline pode conter só fonte e terminal, e isso é bastante comum. O jeito mais seguro de reconhecer uma operação intermediária é pela assinatura: ela sempre devolve um Stream. O tipo do elemento pode mudar ao longo do caminho — um map, por exemplo, pode transformar um `Stream<Livro>` em um `Stream<String>`.

Operação terminal

Todo pipeline precisa terminar, obrigatoriamente, em uma operação terminal — é ela que produz um resultado concreto ou um efeito colateral. `forEach` e `forEachOrdered` são as únicas que retornam void; todas as demais devolvem algum valor: boolean, long, Optional, List, Map... Sem uma operação terminal, o pipeline nunca executa — ele fica parado, como uma receita nunca colocada no forno.

O pipeline como caixa-preta

Por conta da avaliação lazy, o Stream funciona como uma caixa-preta: a fonte entra, o resultado da operação terminal sai, e o que acontece no meio não é garantido acontecer na ordem exata em que foi escrito. Antes de processar qualquer elemento, a implementação de Stream avalia o pipeline inteiro em busca da forma mais eficiente de obter o resultado — podendo reordenar, combinar, ou até eliminar operações intermediárias por completo. O resultado final é sempre consistente, mas o caminho interno não é garantido, ao contrário de um encadeamento comum de métodos, onde cada chamada executa exatamente na ordem escrita.

♦ Regra de ouro — sem efeitos colaterais

Nunca modifique uma variável externa dentro do lambda de uma operação intermediária. Se a otimização decidir pular um filtro — por exemplo, porque a fonte já garante aquela condição — o corpo do lambda nunca roda, e qualquer contador que você esperava incrementar ali simplesmente fica parado.

Pipelines parciais e a regra do uso único

Não é preciso escrever o pipeline inteiro de uma vez: dá para guardar um pipeline parcial, sem operação terminal, numa variável, e só depois encadear o terminal em outra linha do código.

```
var pipelineParcial = catalogo.stream()
    .filter(l -> l.estoque() > 0)
    .sorted(Comparator.comparing(Livro::preco));
// nada foi executado ainda — é só uma descrição do processamento

pipelineParcial.forEach(System.out::println); // agora sim, o pipeline dispara

// pipelineParcial.forEach(l -> System.out.println(l.titulo()));
// ERRO: IllegalStateException – stream já foi operado ou fechado
```

Depois que a operação terminal é chamada, o pipeline abre a válvula, processa os dados e fecha — não há como reabri-lo. Para repetir o processamento sobre os mesmos dados, é preciso montar um novo pipeline a partir da mesma fonte.

Estilo imperativo vs. declarativo

O contraste fica mais claro colocando lado a lado a mesma tarefa escrita das duas formas. No estilo imperativo (Collection), você decide como iterar, filtrar e transformar passo a passo. No estilo declarativo (Stream), você descreve o resultado esperado e deixa o runtime decidir como chegar lá.

```
// Imperativo – controle explícito de cada etapa:
List<Livro> primeiros = new ArrayList<>(catalogo.subList(0, 20));
Collections.sort(primeiros, Comparator.comparing(Livro::titulo));
for (Livro l : primeiros) {
    if (l.estoque() > 0) {
        System.out.println(l.titulo());
    }
}

// Detalhe: primeiros é uma view de catalogo – o sort acima o altera de fato!

// Declarativo – o mesmo resultado, sem tocar na fonte original:
catalogo.stream()
    .limit(20)
    .filter(l -> l.estoque() > 0)
    .sorted(Comparator.comparing(Livro::titulo))
    .forEach(System.out::println);
```

03

Fontes de Streams

Collection, Arrays, Stream.of, iterate, generate, IntStream e Map

O Java oferece oito formas principais de criar um Stream. Duas delas podem gerar streams infinitos: `Stream.generate` e a versão de dois parâmetros de `Stream.iterate`. Vale registrar também que `range` e `rangeClosed` não existem para `DoubleStream` — matematicamente, há infinitos números reais entre quaisquer dois doubles, então um intervalo contável não faz sentido nesse caso.

Método	Finito	Infinito
<code>Collection.stream()</code>	✓	
<code>Arrays.stream(T[])</code>	✓	
<code>Stream.of(T...)</code>	✓	
<code>Stream.iterate(seed, UnaryOperator)</code>	✓	✓
<code>Stream.iterate(seed, Predicate, UnaryOperator)</code>	✓	
<code>Stream.generate(Supplier)</code>		✓
<code>IntStream.range / rangeClosed</code>	✓	

Collection e Arrays como fonte

A forma mais comum de criar um Stream parte de uma `Collection`: qualquer implementação expõe o método `stream()`. Para arrays, o auxiliar estático é `Arrays.stream()`. As duas produzem streams finitos, cujo tamanho já é conhecido de antemão.

```
// A partir de uma Collection:
List<String> generos = List.of("Ficção", "Romance", "Fantasia");
generos.stream()
    .sorted(Comparator.reverseOrder())
    .forEach(System.out::println); // Romance, Fantasia, Ficção

// A partir de um array:
String[] editoras = {"Nortel", "Alameda", "Cerrado"};
Arrays.stream(editoras)
    .sorted()
    .forEach(System.out::println); // Alameda, Cerrado, Nortel
```

Stream.of e Stream.concat

`Stream.of` recebe um número variável de argumentos (varargs) e cria um stream finito diretamente — ótimo para um conjunto pequeno e conhecido de literais. Já `Stream.concat` une dois streams em um só, preservando as operações que cada um já carregava — o que permite montar uma rede de pipelines com processamentos distintos por segmento antes de juntá-los.

```
var nacionais = Stream.of("Torto Arado", "Quarto de Despejo")
    .map(String::toUpperCase); // sem terminal ainda

var importados = Arrays.stream(new String[]{"Dune", "1984", "Circe"})
    .sorted(); // sem terminal ainda

Stream.concat(nacionais, importados)
    .map(titulo -> "» " + titulo)
    .forEach(System.out::println);
// cada segmento manteve as próprias operações até o concat consumir tudo
```

◆ Detalhe importante

As operações de cada stream parcial (`map`, `sorted`) não rodam até que o `concat` receba a operação terminal. Só nesse momento o pipeline inteiro — incluindo cada segmento individualmente — é avaliado e executado de fato.

Stream.generate — infinito via Supplier

`Stream.generate` recebe um `Supplier`, cujo método `get()` não recebe argumento nenhum e devolve um valor novo a cada chamada — indefinidamente. Por isso, um `limit()` (ou outro corte) é obrigatório, sob pena de loop infinito.

```
var random = new Random();

// Simula 10 avaliações aleatórias de 1 a 5 estrelas (stream infinito):
Stream.generate(() -> random.nextInt(1, 6))
    .limit(10)
    .forEach(System.out::println);
```

Stream.iterate — infinito e finito

`Stream.iterate` existe em duas formas. A versão de dois parâmetros (semente + `UnaryOperator`) gera um stream infinito: a semente é o primeiro valor, e o `UnaryOperator` calcula cada próximo a partir do anterior. A versão de três parâmetros adiciona um `Predicate` no meio, o que torna o stream finito — a geração para assim que o predicado devolver `false`.

```
// Infinita (2 parâmetros) – exige limit:
IntStream.iterate(1, n -> n * 2)
    .limit(10)
    .forEach(System.out::println); // 1, 2, 4, 8 ... até o 10º termo

// Finita (3 parâmetros) – Predicate embutido, sem precisar de limit:
IntStream.iterate(1000, codigo -> codigo <= 1050, codigo -> codigo + 10)
    .forEach(System.out::println); // gera os códigos de ISBN de 1000 a 1050
```

♦ Atenção com streams infinitos

O runtime não gera todos os elementos de antemão para só depois aplicar as operações — a avaliação lazy garante que cada elemento é produzido conforme necessário. Ainda assim, sem um operador de corte (limit, takeWhile, ou o Predicate do iterate), o pipeline nunca termina e o programa trava.

IntStream.range e rangeClosed

Para intervalos numéricos de inteiros, `IntStream.range` e `IntStream.rangeClosed` são as formas mais diretas. A diferença: `range` exclui o limite superior, `rangeClosed` o inclui. Os mesmos métodos existem em `LongStream`.

```
// range(1, 51) vai até 50 (exclui 51):
IntStream.range(1, 51).forEach(System.out::println);

// rangeClosed(1, 50) também vai até 50, mas incluindo-o explicitamente:
IntStream.rangeClosed(1, 50).forEach(System.out::println);
```

Map como fonte de Stream

A interface `Map` não implementa `Collection`, então não existe um `stream()` direto nela. Para transformar um `Map` em `Stream`, usamos uma das três views que ele expõe.

View do Map	Tipo resultante	Quando usar
<code>map.keySet().stream()</code>	<code>Stream<K></code>	Só as chaves importam
<code>map.values().stream()</code>	<code>Stream<V></code>	Só os valores importam
<code>map.entrySet().stream()</code>	<code>Stream<Map.Entry<K,V>></code>	Precisa de chave e valor juntos

```
Map<String, Integer> estoquePorGenero = new LinkedHashMap<>();
estoquePorGenero.put("Ficção", 120);
estoquePorGenero.put("Romance", 85);
estoquePorGenero.put("Fantasia", 64);

estoquePorGenero.entrySet().stream()
    .map(e -> e.getKey() + ": " + e.getValue() + " unidades")
    .forEach(System.out::println);
// Ficção: 120 unidades
// Romance: 85 unidades
// Fantasia: 64 unidades
```

Note que o map transformou cada Map.Entry em uma String — o tipo do stream mudou ao longo do pipeline, exatamente como visto no capítulo anterior.

04

Operações Intermediárias — Filtros

distinct, filter, limit, skip, takeWhile e dropWhile

As operações de filtragem são aquelas que alteram a quantidade de elementos que passam pelo pipeline — ao contrário das operações de transformação (como `map`), que atuam em todos os elementos sem remover nenhum.

Operação	Descrição
<code>distinct()</code>	Remove duplicatas, usando <code>equals()</code> do tipo para decidir unicidade
<code>filter(Predicate)</code>	Mantém apenas elementos que satisfazem a condição
<code>takeWhile(Predicate)</code>	Mantém enquanto a condição for <code>true</code> ; para no primeiro <code>false</code>
<code>dropWhile(Predicate)</code>	Descarta enquanto a condição for <code>true</code> ; inclui a partir do primeiro <code>false</code>
<code>limit(long n)</code>	Reduz o stream a, no máximo, n elementos
<code>skip(long n)</code>	Pula os primeiros n elementos

filter e distinct em ação

Um bom exemplo para fixar essas operações é filtrar o catálogo de uma livraria por preço, removendo livros repetidos que aparecem em mais de uma promoção.

```
catalogo.stream()
    .filter(l -> l.preco() <= 60.0) // só livros até R$ 60
    .map(Livro::titulo)
    .distinct() // remove títulos repetidos
    .sorted()
    .forEach(System.out::println);

// Múltiplos filtros também são permitidos — o runtime pode até
// fundi-los internamente em um único passo de avaliação.
```

skip, dropWhile e takeWhile — diferenças que importam

`skip(n)` sempre descarta exatamente n elementos, não importa quais sejam seus valores. `dropWhile` também descarta elementos, mas com base numa condição — e só enquanto ela permanecer verdadeira. Assim que o primeiro elemento falha na condição, o `dropWhile` encerra seu trabalho definitivamente, e todo o restante passa, mesmo que volte a satisfazer a condição depois. É basicamente um mini laço `while` embutido no pipeline.

```
// Catálogo já ordenado por preço, do mais barato ao mais caro:
List<Livro> porPreco = catalogo.stream()
    .sorted(Comparator.comparing(Livro::preco))
    .toList();

// skip – pula sempre os 3 primeiros, sejam quais forem:
porPreco.stream().skip(3).forEach(l -> System.out.println(l.titulo()));

// dropWhile – descarta enquanto o preço for menor que 30, mesmo que
// mais à frente reapareça um livro abaixo de 30:
porPreco.stream()
    .dropWhile(l -> l.preco() < 30.0)
    .forEach(l -> System.out.println(l.titulo()));

// takeWhile – o oposto: mantém enquanto a condição for verdadeira
// e para definitivamente no primeiro que falhar:
porPreco.stream()
    .takeWhile(l -> l.preco() < 30.0)
    .forEach(l -> System.out.println(l.titulo()));
```

♦ dropWhile e takeWhile exigem stream ordenado

Essas duas operações só têm comportamento previsível em streams ordenados. Em fontes sem ordem garantida, a própria documentação da Oracle descreve o resultado como não-determinístico — execuções diferentes podem devolver resultados diferentes. Sempre garanta que a fonte esteja ordenada antes de usá-las.

distinct com Stream.generate

Combinar `distinct()` com `Stream.generate` é um padrão útil para obter um conjunto de valores únicos a partir de uma fonte aleatória — por exemplo, sortear códigos promocionais sem repetição.

```
var random = new Random();

// Sorteia até 30 códigos entre 1000 e 9999, remove repetidos e ordena:
Stream.generate(() -> random.nextInt(10000, 100000))
    .limit(30) // limitar ANTES do distinct permite tamanho variável
    .distinct()
    .sorted()
    .forEach(System.out::println);

// ATENÇÃO: limit antes de distinct = no máximo 30 códigos, com repetições
// já removidas depois. limit DEPOIS de distinct garantiria exatamente 30
// códigos únicos, mas pode demorar bem mais para um espaço amostral pequeno.
```

A API de Streams foi desenhada de propósito para lembrar SQL: `filter` equivale à cláusula `WHERE`, `sorted` ao `ORDER BY`, e `limit` e `distinct` existem com nomes praticamente idênticos em várias linguagens de consulta. Você descreve o que quer obter, e o processador de streams decide como obter — inclusive otimizando ou fundindo operações dentro dessa caixa-preta que vimos no capítulo anterior.

05

Operações Intermediárias — Transformações

map, peek, sorted e streams primitivos

Enquanto as operações de filtragem decidem quais elementos permanecem, as operações de transformação atuam sobre cada elemento presente, podendo alterar seu tipo ou apenas inspecioná-lo sem modificação.

Operação	Descrição
map(Function)	Aplica uma Function a cada elemento; o tipo de saída pode diferir do de entrada
peek(Consumer)	Executa um Consumer em cada elemento sem alterar o stream — uso voltado a debug
sorted()	Ordena em ordem natural; exige que o tipo implemente Comparable
sorted(Comparator)	Ordena usando o Comparator fornecido — necessário para tipos sem ordem natural

map — mudando o tipo ao longo do pipeline

map é a operação de transformação mais versátil: recebe uma Function e pode devolver qualquer tipo R, independentemente do tipo T de entrada. A partir daquele ponto do pipeline, o tipo do stream muda — e é possível encadear vários map em sequência, cada um operando sobre o tipo deixado pelo anterior.

```
// Stream começa como Stream<Integer> (códigos de posição na prateleira)
var etiquetas = IntStream.rangeClosed(1, 50) // IntStream
    .mapToObj(i -> new Livro( // Stream<Livro>
        "Prateleira " + (char) ('A' + i / 10),
        "Vaga " + (i % 10 + 1),
        19.90 + i))
    .map(Livro::titulo); // Stream<String>
// Cada map transforma o tipo: int -> Livro -> String
```

mapToDouble, mapToInt, mapToLong — streams primitivos

Quando o resultado de um mapeamento é um número primitivo, trocar map por mapToDouble, mapToInt ou mapToLong evita o custo de boxing/unboxing e libera operações numéricas extras, como sum(), average() e summaryStatistics(). Para voltar a um Stream genérico depois, usa-se mapToObj() ou boxed().

Stream primitivo	De referência → primitivo	Primitivo → referência
DoubleStream	mapToDouble(ToDoubleFunction)	mapToObj / boxed()
IntStream	mapToInt(ToIntFunction)	mapToObj / boxed()
LongStream	mapToLong(ToLongFunction)	mapToObj / boxed()

```
DoubleStream precos = catalogo.stream()
    .mapToDouble(Livro::preco); // Stream<Livro> -> DoubleStream

OptionalDouble precoMedio = catalogo.stream()
    .mapToDouble(Livro::preco)
    .average(); // só existe em streams primitivos
```

sorted — com e sem Comparator

Tipos que não implementam Comparable — como a maioria dos records de domínio — exigem um Comparator em sorted(). O método Comparator.comparing() aceita uma referência a método como chave de ordenação, e pode ser encadeado com thenComparing() para critérios de desempate.

```
catalogo.stream()
    .sorted(Comparator.comparing(Livro::preco) // mais barato primeiro
        .thenComparing(Livro::titulo)) // empate: por título
    .forEach(System.out::println);
```

peek — uma janela de depuração

peek recebe um Consumer e, portanto, não tem como alterar os elementos do stream — sua finalidade declarada na documentação é dar suporte à depuração. É especialmente útil em pipelines que terminam em operações de redução, como count(), onde não existe um forEach para inspecionar os elementos um a um durante o desenvolvimento.

```
long comEstoqueBaixo = catalogo.stream()
    .peek(l -> System.out.println("Avaliando: " + l.titulo()))
    .filter(l -> l.estoque() < 5)
    .peek(l -> System.out.println("Estoque baixo: " + l.titulo()))
    .count(); // só ao chegar aqui, na operação terminal, o pipeline roda
```


♦ peek não substitui logging de produção

Efeitos colaterais dentro de peek sofrem do mesmo problema discutido no capítulo 02: o runtime pode pular ou reordenar operações intermediárias durante a otimização. Reserve peek exclusivamente para entender o fluxo de dados durante o desenvolvimento — nunca para lógica que precisa rodar de verdade.

06

Operações Terminais — Estatísticas e Matching

count, sum, average, summaryStatistics e allMatch/anyMatch/noneMatch

As operações terminais encerram o pipeline e produzem um resultado ou efeito colateral concreto. Elas se organizam em quatro grandes grupos, mostrados na tabela abaixo — este capítulo cobre estatísticas e matching; o próximo cobre transformação e redução de tipo.

Categoria	Operações
Matching e busca	<code>allMatch</code> , <code>anyMatch</code> , <code>noneMatch</code> , <code>findAny</code> , <code>findFirst</code>
Estatísticas numéricas	<code>count</code> , <code>sum</code> , <code>average</code> , <code>max</code> , <code>min</code> , <code>summaryStatistics</code>
Transformação e redução de tipo	<code>collect</code> , <code>reduce</code> , <code>toArray</code> , <code>toList</code> (capítulo 07)
Processamento	<code>forEach</code> , <code>forEachOrdered</code>

Operações de agregação numérica

Uma operação de redução combina o conteúdo do stream em um único resultado — que pode ser um primitivo (`long`, no caso de `count()`), um wrapper como `Optional`, ou qualquer estrutura de sua escolha.

Operação	Retorno	Disponível em
<code>count()</code>	<code>long</code>	Todos os streams
<code>max / min(Comparator)</code>	<code>Optional<T></code>	Todos os streams
<code>average()</code>	<code>OptionalDouble</code>	Streams primitivos (<code>Int/Long/Double</code>)
<code>sum()</code>	<code>double / int / long</code>	Streams primitivos
<code>summaryStatistics()</code>	<code>*SummaryStatistics</code>	Streams primitivos

```
DoubleSummaryStatistics resumo = catalogo.stream()
    .mapToDouble(Livro::preco)
    .summaryStatistics();

System.out.println(resumo.getCount()); // total de livros
System.out.println(resumo.getSum()); // soma de todos os preços
System.out.println(resumo.getAverage()); // preço médio
System.out.println(resumo.getMin()); // livro mais barato
System.out.println(resumo.getMax()); // livro mais caro
```

Matching — perguntas de verdadeiro ou falso sobre o dataset

Três operações devolvem boolean e recebem um Predicate, permitindo consultar o conjunto de dados como um todo.

Operação	Retorna true quando...
allMatch(Predicate)	TODOS os elementos satisfazem a condição
anyMatch(Predicate)	PELO MENOS UM elemento satisfaz a condição
noneMatch(Predicate)	NENHUM elemento satisfaz a condição

```
boolean todosDisponiveis = catalogo.stream()
    .allMatch(l -> l.estoque() > 0);

boolean algumEmPromocao = catalogo.stream()
    .anyMatch(l -> l.preco() < 20.0);

boolean nenhumEsgotado = catalogo.stream()
    .noneMatch(l -> l.estoque() == 0);
```

Vale notar que allMatch, anyMatch e noneMatch podem interromper o processamento assim que o resultado já está decidido — anyMatch para no primeiro true encontrado, allMatch para no primeiro false. Isso os torna mais eficientes do que contar todos os elementos manualmente quando só a resposta booleana importa.

07

Operações Terminais — Transformações

toList, toArray, collect, reduce, flatMap e Collectors

toList e toArray — materializando o stream

A diferença mais relevante entre as formas de coletar um stream é a mutabilidade do resultado. `toList()` — direto no Stream, desde o Java 16 — devolve uma lista imutável; chamar `add()` nela lança `UnsupportedOperationException`. Para uma lista modificável, use `collect(Collectors.toList())`, que devolve um `ArrayList` de verdade.

Operação	Retorno	Mutável?
<code>toList()</code>	<code>List<T></code>	Não — atalho direto, Java 16+
<code>collect(Collectors.toList())</code>	<code>List<T></code> (<code>ArrayList</code>)	Sim
<code>toArray(T[]::new)</code>	<code>T[]</code>	Sim, e com tipo preservado

```
// Imutável — ótimo quando não há intenção de alterar depois:
List<Livro> maisVendidos = catalogo.stream()
    .filter(l -> l.estoque() < 10)
    .limit(5)
    .toList();

// Mutável — necessário quando o resultado precisa ser embaralhado,
// ordenado in-place ou receber novos elementos:
List<Livro> paraVitrine = catalogo.stream()
    .filter(l -> l.genero().equals("Ficção"))
    .collect(Collectors.toList());

Collections.shuffle(paraVitrine); // funciona — é um ArrayList de verdade
```

A interface Collector e a classe Collectors

Collector não é uma interface funcional — ela define quatro funções que trabalham em conjunto para acumular elementos num container mutável: `supplier` (cria o container), `accumulator` (adiciona um elemento), `combiner` (funde dois containers em processamento paralelo) e `finisher` (transformação final, opcional). A classe `Collectors` traz implementações prontas para os casos mais comuns.

```
import static java.util.stream.Collectors.*;

// groupingBy – Map<gênero, List<livros>>:
var porGenero = catalogo.stream()
    .collect(groupingBy(Livro::genero));

// groupingBy + counting – quantos livros existem por gênero:
var contagemPorGenero = catalogo.stream()
    .collect(groupingBy(Livro::genero, counting()));

// partitioningBy – Map<Boolean, List<livros>>:
var disponibilidade = catalogo.stream()
    .collect(partitioningBy(l -> l.estoque() > 0));
// disponibilidade.get(true) -> livros em estoque

// averagingDouble – preço médio por gênero:
var precoMedioPorGenero = catalogo.stream()
    .collect(groupingBy(Livro::genero, averagingDouble(Livro::preco)));

// joining – concatena títulos com separador:
String titulos = catalogo.stream()
    .map(Livro::titulo)
    .collect(joining(", "));
```

reduce — acumulação em valor único

reduce se diferencia de collect por produzir um único valor, em vez de acumular num container. A versão de dois parâmetros recebe uma semente (identity) — o valor inicial, equivalente à variável total de um laço — e um BinaryOperator que combina os elementos. Ela devolve o tipo T diretamente, nunca um Optional. A versão sem semente devolve Optional, já que o stream pode estar vazio.

```
// reduce com semente – soma o valor total do estoque em reais:
double valorTotalEmEstoque = catalogo.stream()
    .mapToDouble(l -> l.preco() * l.estoque())
    .reduce(0.0, Double::sum);

// reduce sem semente – concatena os gêneros distintos:
Optional<String> generosUnicos = catalogo.stream()
    .map(Livro::genero).distinct().sorted()
    .reduce((acumulado, proximo) -> acumulado + ", " + proximo);

// Na prática, para somas simples, sum() em stream primitivo é mais direto:
double total = catalogo.stream()
    .mapToDouble(l -> l.preco() * l.estoque())
    .sum();
```

flatMap — achatando estruturas aninhadas

flatMap resolve o problema de estruturas aninhadas. Enquanto map troca cada elemento por outro elemento (transformação 1-para-1), flatMap troca cada elemento por um stream de elementos e achata tudo num único stream de saída. Sem flatMap, processar uma lista de pedidos — cada um com sua própria lista de itens — resultaria num inútil Stream<Stream<ItemPedido>>.

```
// Contexto: cada Pedido tem uma List<ItemPedido>
List<Pedido> pedidos = obterPedidosDoDia();

// SEM flatMap – fica um Stream<Stream<ItemPedido>>, difícil de usar:
long semFlatMap = pedidos.stream()
    .mapToLong(p -> p.itens().stream()
        .filter(i -> i.quantidade() > 1)
        .count())
    .sum();

// COM flatMap – um único Stream<ItemPedido>, filtragem direta:
long totalItensMultiplos = pedidos.stream()
    .flatMap(p -> p.itens().stream()) // achata para Stream<ItemPedido>
    .filter(i -> i.quantidade() > 1)
    .count();
```

◆ Quando usar flatMap

Sempre que você se deparar com um `Stream<List<T>>` ou qualquer estrutura aninhada onde é preciso operar diretamente nos elementos internos, `flatMap` é a resposta certa.

08

Optional

Container para valores que podem estar ausentes

Optional é uma classe genérica cujo propósito é servir de container para um valor que pode ou não existir. Ela foi criada para atacar de frente o problema da `NullPointerException`, um dos erros mais comuns do dia a dia em Java. Seu uso primário é como tipo de retorno de método — nunca como campo de instância ou parâmetro.

Optional resolve uma ambiguidade recorrente: quando 'nenhum resultado' é uma situação perfeitamente válida, versus quando é, de fato, um erro. Pense em buscar um livro pelo título num catálogo — não encontrar nada é um resultado normal, não uma falha do sistema. Optional sinaliza exatamente isso: o valor pode estar ausente, e quem chama o método precisa tratar essa possibilidade com segurança.

Métodos de fábrica — nunca use new

Optional não tem construtor público. A regra mais importante da classe: um método que devolve Optional nunca deve devolver null — deve devolver `Optional.empty()`. Fazer o contrário anula completamente o propósito da classe.

Método de fábrica	Comportamento com null	Quando usar
<code>Optional.empty()</code>	N/A — sem valor	Retorno padrão quando não há resultado válido
<code>Optional.of(valor)</code>	Lança <code>NullPointerException</code> na hora	Quando há certeza de que nunca é null
<code>Optional.ofNullable(valor)</code>	Vira <code>Optional.empty()</code> se for null	Quando o valor pode vir de fonte externa

Consumindo um Optional com segurança

O método `get()` existe, mas é arriscado sem checar `isPresent()` antes. As alternativas funcionais a seguir são mais seguras e mais idiomáticas.

Método	Se o Optional está vazio	Quando usar
ifPresent(Consumer)	Não faz nada	Executar uma ação só se o valor existir
ifPresentOrElse(Consumer, Runnable)	Executa o Runnable	Tratar os dois casos num único ponto
orElse(padrão)	Devolve o padrão (sempre avaliado)	Valor padrão barato — literal ou constante
orElseGet(Supplier)	Chama o Supplier só se vazio	Valor padrão caro — construção de objeto
orElseThrow(Supplier)	Lança a exceção do Supplier	Quando a ausência é, de fato, um erro

♦ orElse vs. orElseGet — diferença real de performance

orElse(buscarLivrePadrao()) avalia buscarLivrePadrao() SEMPRE, mesmo quando o Optional já tem valor presente. Se esse método for custoso — uma consulta ao banco, por exemplo — prefira orElseGet(() -> buscarLivrePadrao()), cujo Supplier só é chamado quando realmente necessário.

Operações terminais que devolvem Optional

Diversas operações terminais devolvem Optional justamente porque podem não encontrar resultado num stream vazio ou totalmente filtrado. As operações de streams primitivos devolvem variantes especializadas, como OptionalDouble, OptionalInt e OptionalLong.

```
// findFirst – respeita a ordem do stream:
Optional<Livro> primeiroBarato = catalogo.stream()
    .filter(l -> l.preco() < 25.0)
    .findFirst();

primeiroBarato.ifPresentOrElse(
    l -> System.out.println("Encontrado: " + l.titulo()),
    () -> System.out.println("Nenhum livro abaixo de R$ 25.")
);

// min – equivalente a sorted + findFirst, porém mais eficiente:
catalogo.stream()
    .min(Comparator.comparing(Livro::preco))
    .ifPresent(l -> System.out.println("Mais barato: " + l.titulo()));

// average em stream primitivo – devolve OptionalDouble:
catalogo.stream()
    .mapToDouble(Livro::preco)
    .average()
    .ifPresentOrElse(
        media -> System.out.printf("Preço médio: R$ %.2f%n", media),
        () -> System.out.println("Catálogo vazio.")
    );
```

♦ Desvantagens do Optional

Optional consome mais memória e pode custar um pouco de performance. Ele não é serializável, e usá-lo como campo de instância, parâmetro de método ou variável local frequente não é recomendado. Seu uso correto é, quase exclusivamente, como tipo de retorno de métodos onde a ausência de valor é semanticamente válida.

Com Streams dominados, você processa coleções de dados de forma declarativa e elegante — combinando pipelines otimizados, Optional para segurança contra null e Collectors para transformações poderosas. Essa é a base de qualquer código Java moderno.